

Dungeon Pathfinder

Project Implementation Report

Šehzada Sijarić

Farah Piralić

Asja Smječanin

September 15, 2026

1 Introduction

Dungeon Pathfinder is an interactive, grid-based game where the player navigates from a start point to an exit. The goal is to collect gold while avoiding hazards. We developed this application using the `natID` framework. It allows users to play the dungeon manually and then watch how different computer algorithms solve the exact same layout. The project implements six distinct search algorithms, an interactive quiz for mine tiles, and a system to visualize how these algorithms explore the map.

This report documents the implementation of the Dungeon Pathfinder application, explaining the game mechanics, code structure, user interface, and visualization features.

2 Game Mechanics

The game takes place on a 10×10 tile grid with the following elements:

- **Player start:** The player begins here and moves one tile at a time using the keyboard.
- **Exit tile:** Reaching this tile ends the level. If the player has at least 20 gold, they win. If they have less, the game restarts.
- **Reward tiles:** Give the player +10 gold.
- **Bandit tiles:** Steal 50% of the player's current gold.
- **Mine tiles:** Trigger a quiz. A wrong answer takes away 5 gold; a correct answer lets the player pass safely.
- **Empty tiles:** Have no effect.

Every new level places five rewards, five bandits and five mines on random empty tiles, so no two runs are identical.

3 Implementation and Architecture

3.1 Game Logic and State Management

The main rules of the game are inside the `GameState` class (in `GameState.h`). We designed this to keep the game data separate from the graphics. To manage the dungeon, we use two different grid systems:

1. `actualGrid[GRID_SIZE][GRID_SIZE]`: This integer array stores the real layout of the dungeon. We use constants to identify items: `EMPTY` (0), `PLAYER` (1), `REWARD` (2), `BANDIT` (3), `MINE` (4), and `EXIT` (5).
2. `displayGrid[GRID_SIZE][GRID_SIZE]`: This secondary array controls what the user sees. It supports extra visuals like `PATH_VISUAL` (6) and `EXPLORED_NODE` (7). This lets us draw the pathfinding lines without changing the actual game data.

The `initializeGame` method sets up the level. It uses a random number generator to put the player on the left side and the exit on the right side. Then, the `placeRandomTile` function adds five rewards, bandits, and mines to random empty spots.

The `movePlayer` method handles movement. If the player steps on a reward, they get gold. If they step on a bandit, they lose half their gold. If they step on a mine, the game pauses the logic and opens the quiz window. When the player reaches the exit, the game checks if they have the required 20 gold to win.

We use a structure called `InitialState` to save the map exactly as it was at the start. This allows the algorithms to run on the clean map, even after the player has moved around and collected items.

3.2 Pathfinding Algorithm Implementation

The algorithms are in the `Algorithms.h` file. They are organized into `DungeonAlgorithms` (for standard searches) and `DungeonMDP` (for probability-based search). Each algorithm returns a `SearchResult` containing the path and the nodes visited.

The four cost-aware searches share one cost table, defined in `getMoveCost`: rewards cost 0, empty tiles, the player tile and the exit cost 1, mines cost 8, and bandits cost 15. Because rewards are free and bandits are expensive, the cost-based algorithms naturally detour through gold and around thieves.

- **Breadth-First Search (BFS)**: This uses a queue to explore the grid layer by layer. It finds the shortest path based on the number of steps, but it ignores the gold cost of tiles. The `bfsSearch` function saves visited spots in `exploredNodes`.
- **Depth-First Search (DFS)**: This uses a stack. It explores as far as possible along each branch before backtracking. It is fast but does not guarantee the best path.
- **Uniform Cost Search (UCS)**: Implemented as `dijkstraSearch`, it uses a priority queue ordered by accumulated cost. Since every move has a positive cost, uniform cost search and Dijkstra's algorithm behave identically here, and the function always returns the cheapest route rather than the shortest one.

- **A* Search:** This combines the movement cost with a “Manhattan distance” heuristic (a calculation of the distance to the goal). It balances the cost of the path with the estimated distance to the exit to find the best route efficiently.
- **Greedy Best-First Search:** This only looks at the distance to the goal. It ignores the cost of tiles. It is fast, but it often makes mistakes because it will walk through bandits if that path looks shorter.
- **Markov Decision Process (MDP):** This is in the `DungeonMDP` namespace. It handles uncertainty. It knows that mines have a 70% chance of being safe (if answered correctly). The `MDPSolver` calculates the value of every state to find a policy that maximizes the expected reward.

3.3 The MDP Solver in Detail

Unlike the five graph searches, the MDP does not treat a tile as a single node. A state is the pair (location, gold), with gold tracked up to `MAX_GOLD_TRACKED = 50`, which gives a state space of $10 \times 10 \times 51$ states and four actions per state. The solver runs value iteration with a discount factor `GAMMA = 0.99`, stopping once the largest value change in a sweep falls below `THETA = 0.0001` or after `MAX_ITERATIONS = 5000` sweeps.

Mines are the only stochastic transition: with probability `MINE_SUCCESS_PROBABILITY = 0.7` the quiz is answered correctly and the gold is kept, and with the remaining probability the player loses 5 gold, so the expected value of entering a mine is a weighted average of both outcomes. Empty tiles carry a small negative reward so that wandering is discouraged, rewards are worth far more than the cost of reaching them, and reaching the exit is only valuable once the player holds the 20 gold needed to win. The result is a policy, not just a path: the route the solver draws is the one obtained by following that policy from the start state.

3.4 User Interface Architecture

The user interface begins with the `MainWindow` class, which inherits from `gui::Window`. This window establishes an initial size of 1200×800 pixels through the `gui::Size` constructor. The `setCentralView` method attaches the main application view, and `setResizable(true)` allows window resizing.

The `MainView` class uses a `gui::GridLayout` with one row and one column to contain the `SimulationCanvas`.

The `SimulationCanvas` class, inheriting from `gui::Canvas`, serves as the primary visualization and interaction component. The constructor enables keyboard events and primary clicks through the `gui::Canvas` base class initialization.

Resource loading occurs through the `gui::Image` and `gui::Sound` constructors using resource identifiers like `":player"`, `":reward"`, and `":soundtrack"`.

3.5 Grid Rendering and Visualization

The `drawGameGrid` method draws the map. It calculates the size of the grid dimensions based on the left zone size, applying a 1% margin, then draws grid lines using `gui::Shape`

objects created with `createLines`. For each grid line, a `gui::Point` array defines the start and end coordinates, and `drawWire(td::ColorID::Gray)` renders the line with specified thickness.

The content of the cells is drawn by `drawCellContent`. If the images load successfully, it draws the icons. If they fail to load, it draws colored rectangles as a backup (e.g., green for player, red for mines).

We visualize the algorithms by drawing `EXPLORED_NODE` tiles in blue. The final path is drawn using yellow and orange lines to create a glowing effect.

3.6 Control Panel Implementation

The right-side control panel, rendered by `drawControlPanel`, organizes all user interface elements. The algorithm selection dropdown uses a `gui::Rect` for the main button and individual `gui::Rect` objects for each dropdown item stored in `dropdownItemRects`. The dropdown expands and collapses based on the `dropdownExpanded` flag, with the arrow character changing between “^” and “v” accordingly.

The speed control features a button created with `gui::Shape::createRoundedRect` and `drawFill(td::ColorID::Moss)`. The `speedControlExpanded` flag controls whether the vertical slider appears. The slider itself uses a filled rectangle for the track and a smaller rectangle for the fill level representing current speed. The handle is drawn as a white rectangle with copper wire border.

Control buttons are created through the `drawButton` helper function, which creates rounded rectangles with `gui::Shape::createRoundedRect` and fills them with either moss green when enabled or dim gray when disabled. Each button’s rectangle is stored for hit detection in mouse event handling.

The algorithm comparison table dynamically displays information about the currently selected algorithm. When an algorithm is selected, the `drawAlgorithmDetails` method shows its name, description, heuristic information, time complexity, space complexity, and execution time using formatted strings and `gui::DrawableString::draw`.

3.7 Quiz Popup System

The quiz is handled in `QuestionsPopUp.h`. We created a `DialogLogin` class based on the standard `gui::Dialog`.

- **Text:** A `gui::Label` shows the question.
- **Input:** The answers are mapped to standard buttons: `ID::OK`, `ID::Apply`, and `ID::Cancel`.
- **Logic:** The `onClick` function checks if the button pressed matches the correct answer and shows an alert with the result.

The questions themselves are drawn from a pool of artificial intelligence and machine learning questions, so the hazard doubles as a short quiz on the course material. If the answer is correct, the game continues; if not, the player is punished by negative 5 gold.

3.8 Player Interaction and Events

Keyboard: The `onKeyPressed` method checks for arrow keys or WASD keys. It sends the direction to `gameState.movePlayer` to update the player's position.

Mouse: The `onPrimaryButtonPressed` method detects clicks. It checks if the user clicked on buttons, the speed slider, or the dropdown menu. If an algorithm is selected, it calls `runAlgorithm`.

3.9 Animation and Visualization System

The visualization system makes the computer's search visible. It works in two phases inside the `updateAnimation` method:

1. It reveals the `exploredNodes` (blue tiles) one by one.
2. Once exploration is done, it draws the `fullAlgorithmPath` (yellow line).

This runs on a timer based on `animationSpeed`. There is also a `stepAnimation` method that lets the user advance one step at a time manually.

3.10 Sound System Integration

We use `gui::Sound` for audio. Effects play when specific events occur, such as collecting a reward or hitting a mine. The soundtrack loops continuously in the background using the `playSoundtrack` method, as long as the game is active.

3.11 Resources and Translations

All user-visible text, images and sounds are loaded through the `natID` resource system rather than being hard-coded. The `res/DevRes.xml` file declares the artwork and translation configurations, and `res/main.xml` holds the interface definition. Translations live in `res/tr/main_XX.xml`, one file per language, and the application currently ships in eight: English, Bosnian, German, French, Spanish, Russian, Turkish and Chinese. The language is read from the stored application properties at startup in `main.cpp`, so the chosen translation persists between sessions.

3.12 Project Structure and Build

The repository follows the layout required for submission: `Docs/` contains this report, and `Implementation/` contains everything needed to build the game, namely `src/` for the sources, `res/` for resources and translations, and `installer/` for the packaging configuration. The build is driven by CMake: `CMakeLists.txt` sets the solution name and pulls in the SDK's `Common.cmake` and `natGUI.cmake`, while `dungeonGame.cmake` collects the sources and links the `mainUtils` and `natGUI` libraries. Because every path is resolved relative to the CMake list directory and the SDK is located through the user's home folder, the project builds unchanged on Windows, macOS and Linux. Ready-made installers for all three platforms are produced with the SDK's own `SetupCollector` from `installer/dungeonGame.xml`.

3.13 Team Distribution

To make the process of working on the project easier, we decided to somewhat correct our initial division of tasks. Each of the students contributed to the project equally. We worked together to come to the solution. Here are our specific contributions for building the project:

- Šehzada Sijarić (19964): Implementing the grid layout and tile rendering, game state logic with pop-up quiz, and integrating the keyboard movements.
- Farah Piralić (20106): Implementing the control panel, visualizing the algorithm movements, algorithm dropdown, control buttons, and comparison table.
- Asja Smječanin (20012): Implementing the logic of BFS, DFS, Dijkstra algorithm, Greedy Best-First Search and A* Search. Making a custom MDP for solving the grid game. Overseeing final system integration and validation.

4 Conclusion

The Dungeon Pathfinder project successfully creates a platform for comparing pathfinding algorithms. By separating the game logic from the visualization, users can experience the dungeon manually and then see how algorithms like A* or MDP solve the challenge. Running all six algorithms on the same saved starting layout makes the differences between them easy to observe: BFS and DFS ignore danger, the cost-based searches route around bandits, and the MDP weighs the risk of a mine against the gold it protects. The application provides clear visual feedback and intuitive controls.

The source code of the project can be found at https://github.com/ssijaric1/ProjAI_DungeonG_Piralic_Smjecanin_Sijaric_.